

Evaluating the Development Models of Two Programming Languages

Part 1: Code Exploration

Both scripts do a simple task. They remove repeated names from a given list and sort them alphabetically. After a quick review, neither script crashed or threw errors when running, which suggested that both worked as intended. However, when looking into the outputs, they show two issues: some duplicates remain, and the ordering is inconsistent. This is a form of “silent semantic failure”, when the execution of technically correct code produces incorrect outputs with potential ethical, legal, and organisational consequences that are often unnoticed (Lou, Jing, and Huang, 2022).

In the initial list, neither script finds any exact duplicates to remove, but both fail to address semantic duplicates such as case variants of the same name. The JavaScript code translates the array input to a Set and back to an array through a call to `sort()`. The Python code translates the array to a set and back to a list through a call to `sort()`. Both scripts treat “Alice” and “alice” as distinct values, although they refer to the same person in most practical applications. The set functions in JavaScript and in Python are case sensitive in comparison operations. This default prioritises speed and predictability. However, it fails for identity data with the needs of case-sensitive comparison.

For sorting, both languages use code-point comparison, not locale-based collation, for the same reason of speed and determinism. Uppercase letters have lower Unicode values compared to the lowercase ones. In the example, we could see

“BOB” coming before “alice” which diverges from common expectations of alphabetical order.

Table 1: Comparison of initial list, script outputs and expected results

Initial List	Script outputs	Expected normalized results
["Alice", "bob", "alice", "Charlie", "BOB", "dave", "Eve", "charlie"]	["Alice", "BOB", "Charlie", "Eve", "alice", "bob", "charlie", "dave"]	["alice", "bob", "charlie", "dave", "eve"]

The table shows clearly the problem; the scripts return eight entries in either language, while only five are expected.

The small adjustment I am proposing is limited to lowercasing names before deduplication. More precise solutions exist, such as dedicated data structures, library functions or locale-aware sorting, but the brief required a light modification. Even this minor change has an impact, since “Alice” and “alice” are now considered as one identity. Oztoprak et al. (2024) present those forms of defensive programming in resource-restricted environments. For the exercise, we assumed case-insensitive names, although case sensitivity remains essential in contexts such as passwords, usernames and file paths. Basic case conversion is not universal either. The lowercase of “I” in Turkish is “i” (dotless “i”), so “name.lower()” is not going to work correctly with Turkish names. Even the most elementary solution carries hidden presumptions.

This is how this simple modification shows in a revised Python script with case normalisation before deduplication:

```
def process_names(names):  
  
    # MODIFICATION: Lowercase to remove duplicates and consistent  
    alphabetical ordering  
  
    normalized = [name.lower() for name in names]  
  
    unique = list(set(normalized))# Deduplicate then convert to  
    list  
  
    unique.sort()# Unicode code-point order  
  
    return unique  
  
# Output: ['alice', 'bob', 'charlie', 'dave', 'eve']
```

And, with the same logic in JavaScript:

```
function processNames(names) {  
  
    // MODIFICATION: Lowercase to remove duplicates and consistent  
    alphabetical ordering,  
  
    let normalized = names.map(name => name.toLowerCase());  
  
    let unique = [...new Set(normalized)];// Spread converts Set  
    to array  
  
    unique.sort();// Unicode code-unit order  
  
    return unique;  
  
}
```

```
// Output: ['alice', 'bob', 'charlie', 'dave', 'eve']
```

Both modifications incorporate lowercase conversion before performing set operations to eliminate semantic duplicates while retaining the functionality.

For less experienced developers, Python’s transformation steps remain visibly linear, making the intent of each operation easier to follow during review and modification. The JavaScript example gives the same result, but by using the spread operator and arrow functions to compress operations into a single expression, increasing the effort required to interpret during the review process. Across both scripts, the reliance on case-sensitive set operations and basic code-point sorting means that semantic duplicates and ordering irregularities can pass through the pipeline without being flagged. These shortcomings stem from the particular logic chosen and from the languages’ default behaviours, rather than from any inherent strength or weakness in either Python or JavaScript.

Stephens (2022) notes that consistent coding practices often reduce maintenance effort more than language features. In this example, the original scripts returned eight entries because the case variants were kept, while the modified versions produced five normalised names (“alice”, “bob”, “charlie”, “dave”, “eve”). The change closes the semantic gap and shows how a small convention can correct the behaviour of a simple processing step.

Part 2: Language Comparison Report

Design philosophy affects how languages implement software quality and cooperation. The upcoming analysis compares and contrasts Python and JavaScript along four axes: programming models, readability, error handling, and use in modern computing.

2.1 Programming Models

Both languages accommodate procedural, object-oriented, and functional approaches, but they support these paradigms differently. Python favours clarity over concision. The Python world encourages readability over the possibility of implicit behaviours. It influences the language's syntax and library design, although individual coding style still varies. JavaScript emphasises flexibility and rapid iteration, supported by dynamic typing and a prototypal object model. Codebases often mix paradigms within a single file, depending on team practices and tooling.

Python's object-oriented programming model depends on class inheritance methods provided through method resolution order, providing reproducible, traceable linkages. JavaScript's prototype chain provides great expressiveness and flexibility, but can result in unforeseen runtime behaviour when an object changes, via adding, removing, and hiding properties. Paradigm mixing in distributed teams will worsen the problem created by weak verification methods (Zhao et al., 2024). In shared code, expressive features can encourage patterns that are hard to test and increase the risk of defects escaping testware. Python's explicit style can reduce some error classes. In JavaScript, comparable assurance relies more on shared discipline and

review. There is a particular benefit in disciplined code practices learned from these syntax strictures that are transferable to other programming languages.

The data-transformation logic in Part 1 provides a useful entry point for considering how each language supports functional programming and its impact on software quality and maintainability. Python highlights the capabilities of first-class functions, lambda functions, and comprehensions as tools of declarative data manipulation in a way that is still readable and maintainable. The capabilities of JavaScript's functional programming, such as arrow functions and methods of manipulation of arrays via `map()`, `filter()`, and `reduce()` methods, are similar and distinct in their ergonomic and syntactic philosophies. The programming paradigm has implications for the types of software errors and their manifestations, as Python's more explicit idioms and widespread linting/type-hinting practices can surface specific errors earlier, whereas JavaScript's flexibility can defer some issues to runtime without strong tooling.

2.2 Readability and Developer Accessibility

Readability sits at the heart of how Python was designed. This was deliberate, and it has practical consequences. Stephens (2022) emphasises that readable code directly supports software quality by reducing errors and easing maintenance burdens. System transparency builds trust, a principle extending from interface design to code architecture. Readable code builds confidence, particularly for developers maintaining systems they never wrote. Python gets there through enforced indentation, meaningful whitespace, and a standard library that favours clarity over cleverness. JavaScript offers more syntactic freedom. That freedom

speeds things up but raises variability, which teams counter through naming conventions, code reviews, and shared style guides. Community standards discourage cryptic shortcuts and promote self-documenting constructs that communicate intent to maintainers who may encounter the code months or years later.

JavaScript, by contrast, permits multiple syntactic approaches to the same operation, creating inconsistency across codebases that can significantly impede comprehension and maintenance. In these scripts, sorting is clearly deterministic but not aligned with user expectations for names: the case-variant ordering in Table 1 shows how “valid-looking” output can still be semantically wrong. A simple demonstration is that sorting `[10, 2, 1]` without a comparator yields `[1, 10, 2]`, illustrating how implicit conversions can surprise maintainers. These implicit behaviours demand vigilance. Such behaviours can introduce subtle bugs in code that otherwise appear correct. Left unchecked, they can accumulate technical debt, as each undocumented workaround adds cognitive load for future maintainers. Code clarity limits downstream errors and simplifies long-term maintenance, making Python's explicit approach particularly valuable in enterprise contexts where multiple teams share responsibility for critical systems. Walton (2024) argues that accessible code serves inclusive design principles, enabling broader participation in software development and audit processes.

2.3 Error Handling and Debugging

Neither code produces errors while handling the ambiguous identity information. In both cases, the code silently accepts the input and generates results that look valid but are semantically incorrect. Many harmful bugs arise not from crashes but from plausible results that mask incorrect assumptions. Python explicitly utilises exception-handling mechanisms through try-except constructs that promote the practice of anticipating failure scenarios and handling them well. JavaScript's methodologies, particularly with async code, are hidden behind promises and async-awaits that might mask the failure causes. In cases of failure during production deployment, sometimes the entire context might not be available for debugging. However, browser debugging tools offer instant and visible feedback that accelerates development cycles within the frontend development paradigm compared to server-side debugging via logging.

The exception handling system in Python provides finer levels of error classification, and this is useful for catching specific errors and also for passing on unexpected errors for higher-level handlers. The use of the else statement in try-catch handlers makes it easier to demarcate error handlers and success handlers in a cleaner way. The try-catch-finally constructs in JavaScript appear similar to those in Python, but they do not provide an else clause, which would be helpful in providing a better demarcation between error code and success code. In both languages, try-catch-finally can clarify control flow when used consistently. According to Stephens (2022), disciplined error handling is essential for good-quality software in production.

2.4 Suitability for Modern Computing Tasks

In data science, analytics, and AI, Python is widely used, and the 2024 GitHub Octoverse report confirms this trend: in 2024, Python overtook JavaScript as the most popular programming language on GitHub, a shift driven largely by increased activity in AI and data-science projects (GitHub, 2024). Its simplicity makes it easy for data analysts to understand, even if they have limited programming experience. In fact, the ease of programming in Python makes it the language of choice in developing hypotheses in the process of building machine learning models. JavaScript, on the other hand, suits web and real-time solutions that can benefit from its event-driven programming model's inherent advantages.

Suitability goes beyond technical ability to issues of governance and auditability, which are becoming ever more critical in regulated sectors. The lecturecast for Unit 12 illustrates the importance of language ecosystems, not just syntax, especially with the evolving role of AI-powered tools, impacting the way developers work with their code (University of Essex Online, 2024). When viewed as part of a broader software tool, these scripts show how language defaults affect governance, auditability, and user impact beyond isolated functions. While Python's clearer syntax can reduce some classes of semantic error, the widespread use of JavaScript in interface-based systems means that flexibility, with proper governance, can surface issues early through direct user engagement. Hence, the suitability of Python applies to regulated, highly audited areas where the integrity of data is of prime importance. JavaScript is suited to rapid, interface-centred development when supported by governance and robust testing to counter the lax approach.

Part 3: Professional Reflection

Issues related to the code have ethical undertones that range into the legal and professional arenas. Participation in the code exploration activity improved my insight into these linkages and is worth investigating.

3.1 The Importance of Readable and Ethical Code

What the tutor scripts did was to unsettle some underlying beliefs that I held about code quality. Readability had been perceived largely from an organisational point of view, and it did not appear to matter. This changed my understanding of the importance of code readability, from affecting real people whose data systems process information to scripts that fail in ways that obscure accountability, often only revealing issues after harm has happened. Fu, Hadid, and Damer (2025) have picked out transparency and interpretability as key ethical issues in computing. Their analysis aligns clearly with the issues raised in the tutor scripts. The tutor scripts are simple, but they reflect issues in complex systems where small errors spread quickly. Small defaults mirror design choices that can escalate into serious identity-record errors in operational systems.

The ACM/IEEE Code of Ethics sets duties to ensure software is safe, conforms to specifications, has been tested and avoids reducing quality of life (Gotterbarn, Miller and Rogerson, 1997). Applying this to the tutor scripts, I see the necessity of this requirement as it passes the tests, but decreases the quality of life by fragmenting identity records of data users. Previously, I saw code readability as an internal quality attribute, but when viewed through the ACM/IEEE Code of Ethics, it serves as an

ethical safeguard, helping identify, question, and fix harm more easily. The focus on Python code readability is not just aesthetic but also a move towards accessible design, making code easier to understand, test, and improve by many. I now view code readability as an ethical safeguard, not just a stylistic choice.

3.2 Issues of Bias, Privacy, or Misuse in Programming

Case sensitivity revealed a weakness in my own thinking: technical defaults embody cultural bias. Case sensitivity is not inherently biased, but it can create biased outcomes when systems ignore linguistic variation. Unicode sort order isn't either. As Walton (2024) writes about culturally responsive computing, these hidden defaults act as devices of exclusion: they systematically disadvantage others whose needs were not taken into account.

The Turkish 'ı' example shows that even corrective measures, such as naïve case normalisation, embed linguistic assumptions that remain invisible until they are deployed across cultures. Real-world involvement in the field of software development has already shown that universal solutions can favour one group of users while working adversely for another. The exercise left unstated points that should have been addressed directly.

According to Wang et al. (2024), algorithmic decision-making can embed discrimination through cumulative design choices, even when no individual decision is explicitly discriminatory, thereby placing the tutor scripts in a broader context.

Kelly (2021) shows the impact of the 2020 A-Level algorithm in the UK affected students unequally, adjusting grades based on school history rather than individual performance (pp. 727-728). The European Union Agency for Fundamental Rights (2022) describes algorithmic bias as generating feedback loops where inequity compounds, yet no single party bears direct responsibility. Reading this sharpened my grasp of how technical choices slot into larger patterns of harm. I would question technical defaults more critically, paying attention to the cultural and linguistic assumptions they contain.

3.3 How Small Coding Choices Can Have Real-World Business or Legal Impact

Looking at these scripts through regulation was eye-opening; I underestimated what's at stake. GDPR's accuracy principle, under Article 5(1)(d), requires personal data to remain current when operational context demands it (European Parliament, 2016). For example, in a customer identity system, case-sensitive fragmentation can create duplicate records that undermine the GDPR's accuracy principle, yet remain operationally invisible until challenged through subject access or erasure requests. None of these issues calls attention to itself; it is business as usual.

GDPR Article 17 (Right to Erasure) creates specific risks I found genuinely concerning upon reflection (European Parliament, 2016). If a user named "Alice" exercises her right to deletion, a system failing to link "Alice" with "alice" may leave records behind, constituting a verifiable breach. I had assumed I understood GDPR, but the issue of accuracy proved not just essential but equally challenging.

Haug, Zachariassen, and van Liempd (2011) list 23 categories of costs related to poor data quality, showing how bad data causes economic and social harm through decreased customer satisfaction and suboptimal decision-making. Financial institutions are also vulnerable to such risks. A bank that fails to merge identical customer records because of case handling issues can misjudge credit risk and make incorrect lending decisions. The GDPR's upper-tier fine of €20 million or 4% of global turnover (European Parliament, 2016) gives a clear incentive, while Li and Zhang (2025) add weight to this by showing how data cleanliness affects risk prediction accuracy. These risks were no longer abstract; this exercise made them concrete. I would evaluate minor coding decisions based on their regulatory and organisational impacts, not just functional correctness.

3.4 Reflecting on Governance Frameworks

The activity made it clearer how these frameworks view this category of risk as a problem to be addressed. NIST (2023) organises risk management into four domains: governance, mapping, measurement, and management. The latent semantic flaw identified in these scripts is a perfect match to this category of risk that these frameworks are meant to highlight. Understanding these frameworks by reading about them after engaging in this process of code exploration brought together ideas from high-level theory that had not been connected before.

As far as general standards go, the EU AI Act requires high-risk AI systems to follow documented quality-management processes and maintain human oversight

throughout the system lifecycle (European Parliament, 2024). Zhao et al. (2024) show how data replica differences in edge systems can create harmful effects that become visible only after deployment. These readings reinforced the point that governance work is substantive rather than an administrative burden. In this particular situation, the ACM/IEEE Software Engineering Code of Ethics is applicable. The Code states that software engineers should not use their signatures in sponsoring software that diminishes the quality of life, bringing harm to the public (Principle 1). In addition, the Code of Ethics puts the responsibility of error detection, correction, and reporting in the software and its documentation on the shoulders of the software engineers (Principle 6) (Gotterbarn, Miller, and Rogerson, 1997). Professional responsibility, I now recognise, encompasses not just apparent failures but latent semantic defects of the kind examined here.

3.5 Synthesis and Professional Implications

Connections between language defaults, discrimination risk, readability, ethical accountability, case sensitivity and GDPR compliance had not been visible to me before this exercise. The tutor scripts were superficial and harmless upon earlier observation. Repositioning them as identity management systems brought out assumptions implicit in them. Wang et al. (2024) provide a classification framework that clarifies how these issues accumulate and interact. Accumulated technical choices can create discriminatory outcomes that are neither deliberate nor obvious, as seen in the case-sensitivity issues that are apparent in the scripts provided. Reading about distributed systems (Oztoprak et al., 2024; Zhao et al., 2024) helped me understand how small choices propagate through complex systems. Moving forward, the plan is to apply a more systematic, rigorous approach towards

understanding identity handling code. This involves questioning the implications of default actions, determining the ways in which these actions may negatively impact particular individuals, as well as considering the ways in which silent computation errors extend throughout the system. The ACM/IEEE Code of Ethics offers a structured way to navigate these competing priorities.

AI Acknowledgement: Generative AI tools (Grammarly and Copilot) were used for proofreading and clarity refinement. The intellectual content remains entirely my own work.

References

- European Parliament (2016) *Regulation (EU) 2016/679 of the European Parliament and of the Council (General Data Protection Regulation)*. *Official Journal of the European Union*, L119, pp. 1–88. Available at: <https://eur-lex.europa.eu/eli/reg/2016/679> (Accessed: 14 December 2025).
- European Parliament (2024) *Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act)*. *Official Journal of the European Union*, L 1689, 12 July. Available at: <https://eur-lex.europa.eu/eli/reg/2024/1689> (Accessed: 20 December 2025).
- European Union Agency for Fundamental Rights (FRA) (2022) *Bias in Algorithms: Artificial Intelligence and Discrimination*. Vienna: European Union Agency for Fundamental Rights. Available at: <https://fra.europa.eu/en/publication/2022/bias-algorithm> (Accessed: 20 December 2025).
- Fu, B., Hadid, A. and Damer, N. (2025) ‘Generative AI in the context of assistive technologies: Trends, limitations and future directions’, *Image and Vision Computing*, 154, 105347. Available at: <https://doi.org/10.1016/j.imavis.2024.105347>.
- GitHub (2024) *Octoverse: AI leads Python to top language as the number of global developers surges*. Available at: <https://github.blog/news-insights/octoverse/octoverse-2024/> (Accessed: 17 January 2026).
- Gotterbarn, D., Miller, K. and Rogerson, S. (1997) ‘Software engineering code of ethics and professional practice’, *Communications of the ACM*, 40(11), pp. 110–118. Available at: <https://doi.org/10.1145/265684.265699>.
- Haug, A., Zachariassen, F. and van Liempd, D. (2011) ‘The costs of poor data quality’, *Journal of Industrial Engineering and Management*, 4(2), pp. 168–193. Available at: <https://doi.org/10.3926/jiem.2011.v4n2.p168-193>.
- Kelly, A. (2021) ‘A tale of two algorithms: The appeal and repeal of calculated grades systems in England and Ireland in 2020’, *British Educational Research Journal*, 47(3), pp. 725–741. Available at: <https://doi.org/10.1002/berj.3705>.
- Li, P. and Zhang, L. (2025) ‘Application of big data technology in enterprise information security management’, *Scientific Reports*, 15, 1022. Available at: <https://doi.org/10.1038/s41598-025-85403-6>.
- Lou, C., Jing, Y. and Huang, P. (2022) ‘Demystifying and checking silent semantic violations in large distributed systems’, *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI '22)*, pp. 91–107. Available at:

<https://www.usenix.org/conference/osdi22/presentation/lou-demystifying>
(Accessed: 15 December 2025).

National Institute of Standards and Technology (NIST) (2023) *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. Gaithersburg, MD: National Institute of Standards and Technology. Available at: <https://doi.org/10.6028/NIST.AI.100-1>.

Oztoprak, A., Hassanpour, R., Ozkan, A. and Oztoprak, K. (2024) 'Security challenges, mitigation strategies, and future trends in wireless sensor networks: a review', *ACM Computing Surveys*, 57(4), Article 104. Available at: <https://doi.org/10.1145/3706583>.

Stephens, R. (2022) *Beginning Software Engineering*. 2nd edn. Indianapolis: John Wiley & Sons.

University of Essex Online (2024) 'Future Trends and Innovations in Computing' [Lecturecast]. *Launch into Computing October 2025 B*. University of Essex Online. Available at: <https://online.essex.ac.uk> (Accessed: 14 December 2025).

Walton, D.J. (2024) *Culturally Responsive Computing: An Introduction into Computer Science, Security, and Technology*. Haverhill, MA: ROTEL (Remixing Open Textbooks with an Equity Lens) Project. Available at: <https://rotel.pressbooks.pub/ciswrcr/> (Accessed: 15 December 2025).

Wang, X., Wu, Y.C., Ji, X. and Fu, H. (2024) 'Algorithmic discrimination: examining its types and regulatory measures with emphasis on US legal practices', *Frontiers in Artificial Intelligence*, 7, 1320277. Available at: <https://doi.org/10.3389/frai.2024.1320277>.

Zhao, Y., Qu, Y., Xiang, Y., Uddin, M.P., Peng, D. and Gao, L. (2024) 'A Comprehensive Survey on Edge Data Integrity Verification: Fundamentals and Future Trends', *ACM Computing Surveys*, 57(1), Article 8. Available at: <https://doi.org/10.1145/3680277>.